
Compsys

BA4 • 2025-2026

Contents

1	Lecture 11 — File Systems	3
1.1	Motivation : Persistance	3
1.2	Abstractions	3
1.3	File Descriptors	4
1.4	Layout Disque	4
1.5	Allocation de Blocs	4
1.6	Coût des Opérations Fichier	5
	Résumé	7

CHAPTER 1

Lecture 11 — File Systems

1.1 Motivation : Persistence

Persistence

État qui **survit au processus qui l'a créé** : terminaison du processus, redémarrage, coupure de courant. Sans cela, tout réside en DRAM (volatile) et disparaît à l'extinction.

Un file system donne aux utilisateurs quatre capacités clés sur le stockage persistant : **Nommer** — **Organiser** — **Partager** — **Protéger**.

1.2 Abstractions

Inode

Structure on-disk représentant un fichier ou dossier : **métadonnées** (taille, propriétaire, permissions, timestamps) + **pointeurs de blocs** vers les données.

Répertoire

Fichier spécial dont le contenu est un tableau de paires (**nom**, **numéro d'inode**). Un flag dans l'inode restreint l'API (pas de `write()` direct). Toute l'arborescence repose sur *une seule* abstraction : le fichier.

Key Takeaway

Résolution de chemin `/tmp/test.txt` : inode racine → data racine → inode `tmp` → data `tmp` → inode `test.txt` → blocs de données. **5 lectures disque minimum.**

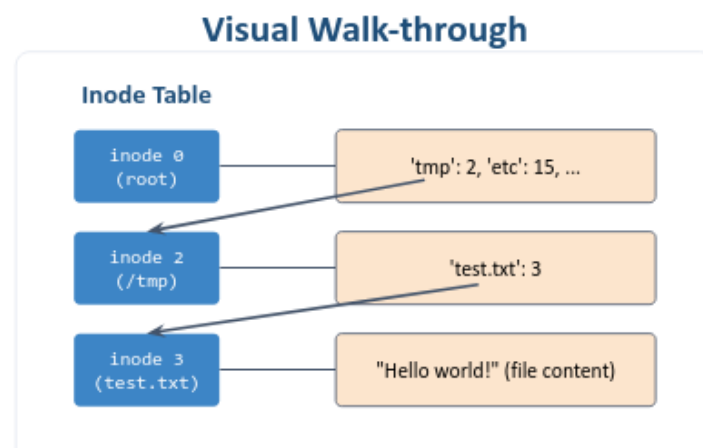


Figure 1.1: Walk de chemin via la table d'inodes.

1.3 File Descriptors

Warning

Refaire le walk à chaque `read()` coûte $250,000 \times 5 = 1,25$ M lectures disque pour un fichier de 1 Go lu en blocs de 4 Ko.

File Descriptor (fd)

Entier par-processus. À l'`open()`, l'OS fait le walk *une seule fois*, met l'inode en cache, et renvoie un fd. Les `read()/write()` suivants utilisent directement ce cache. fd 0/1/2 = STDIN/STDOUT/STDERR.

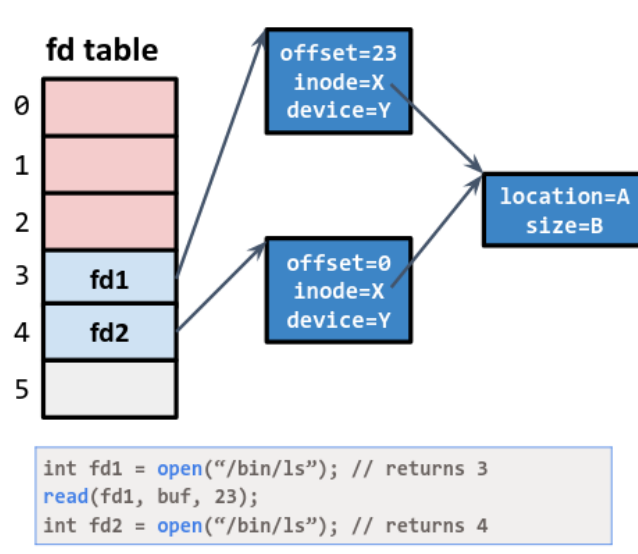


Figure 1.2: Table de fd par processus. fd1 et fd2 partagent le même inode mais ont des offsets indépendants.

1.4 Layout Disque

Superblock

Métadonnées de la partition : nombre d'inodes/blocs, emplacement de la table d'inodes, bitmaps libres. Lu en premier au montage ; sa corruption rend la partition illisible.

`mount <device> <dir>` rattache la racine d'un système de fichiers à un point de montage existant. Tout répertoire peut être un point de montage.

1.5 Allocation de Blocs

Méthode	Avantage	Inconvénient
Contiguous	Lecture séquentielle rapide	Fragmentation, extension coûteuse
Linked list	Extension/suppression facile	Accès aléatoire $O(n)$
FAT	Liens hors des blocs	Table entière en mémoire
Multi-level	$O(1)$ petits fichiers, ≤ 4 I/O grands	Léger overhead très grands fichiers

File system layout on the disk

- File system is stored on disks
 - Disk can be divided into one or more partitions
 - Sector 0 of disk: master boot record (MBR), which contains:
 - Bootstrap code (loaded and executed by the firmware)
 - Partition table (addresses of where partition start and end)
 - First block of each partition has a boot block
 - Loaded by executing code in MBR and executed on boot



19

Figure 1.3: MBR (secteur 0) → table de partitions → chaque partition : **boot block** | **superblock** | **bitmap espace libre** | **table d'inodes** | **blocs de données**.

Multi-Level Indexing (Unix)

L'inode contient : 12 pointeurs **directs** + 1 **simple-indirect** + 1 **double-indirect** + 1 **triple-indirect**. Avec blocs de 4 Ko et pointeurs de 4 octets (1024 ptr/bloc) :

$$12 \times 4 \text{ Ko} = 48 \text{ Ko} \quad 1024 \times 4 \text{ Ko} = 4 \text{ Mo} \quad 1024^2 \times 4 \text{ Ko} = 4 \text{ Go} \quad 1024^3 \times 4 \text{ Ko} = 4 \text{ To}$$

1.6 Coût des Opérations Fichier

Nombre d'opérations disque

- **open()** (chemin à 2 niveaux) : 5 lectures.
- **read()** (n^{ème} bloc) : 1 lecture (data) + 1 écriture (inode atime). Premier appel : +1 lecture inode.
- **write()** (nouveau bloc) : 2 lectures + 3 écritures (bitmap×2, inode, data).
- **create()** : +~6 ops supplémentaires (bitmap inode, nouvel inode, data dir, inode dir).

Block Cache

Le cache disque (DRAM) élimine la majorité des I/O. Taux de hit > 99% fréquents ⇒ jusqu'à **1000×** de gain. C'est pour cela qu'éviter les I/O est la priorité numéro 1.

4. Multi-level indexing

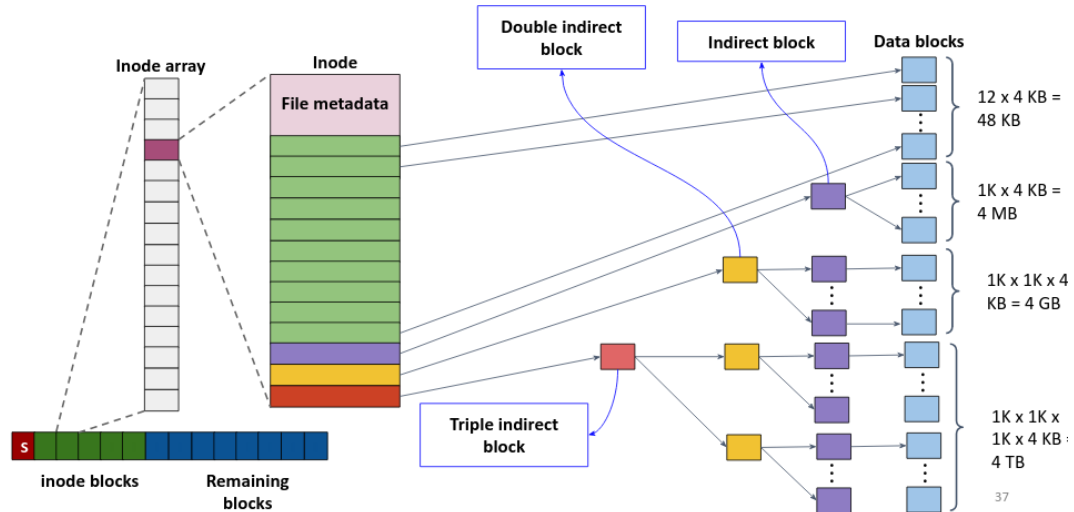


Figure 1.4: Structure multi-niveau. Les petits fichiers n'utilisent que les pointeurs directs ; les grands fichiers traversent jusqu'à 3 niveaux d'indirection (4 lectures disque max).

Sizing the multi-level index tree

- **Setup:** Block size = 4 KB, pointer size = 4 bytes
- **Pointers per block:** 4 KB / 4 B = 1024 pointers per indirect block
- 12 direct pointers → 12 x 4 KB = 48 KB
- 1 single-indirect → 1024 x 4 KB = 4 MB
- 1 double-indirect → 1024² x 4 KB = 4 GB
- 1 triple-indirect → 1024³ x 4 KB = 4 TB
- Worst-case lookup: ≤ 4 disk reads, even for the largest file. Small files pay zero indirection cost.

Figure 1.5: Calcul des tailles maximales (blocs 4 Ko, pointeurs 4 octets : 1024 ptr/bloc).

Résumé

Key Takeaway

- **Persistence** : état qui survit au processus. DRAM volatile $\neq$ disque non-volatile.
- **4 buts FS** : Nommer, Organiser, Partager, Protéger.
- **Inode** : métadonnées + pointeurs de blocs. Un inode par fichier/dossier.
- **Répertoire** : fichier dont le contenu est un tableau (`nom`, `inode`).
- **fd** : entier par-processus cachant (`inode`, `offset`) après `open()`. fd 0/1/2 = STDIN/STDOUT/STDERR.
- **Path walk** : 5 lectures pour `/tmp/test.txt` — coûteux si répété à chaque `read()`.
- **Layout partition** : boot | superblock | bitmap | inodes | data.
- **Multi-level indexing** : 12 directs (48 Ko) + single (4 Mo) + double (4 Go) + triple (4 To). ≤ 4 lectures extra au pire.
- **Block cache** : hit rate $> 99\% \Rightarrow 1000\times$ speedup. Éviter les I/O est critique.